CalcDb: the Access database engine, rebuilt

Draft 2 · 2026-11-10 · working document for review · CHARTER.md has the detail

What it is

CalcDb is the database engine of DNA Calc: a clean-room Rust reimplementation of Jet 4 and ACE, with the DAO and ADO object models on top. VBA, `.xll` and .NET add-ins that use ACE in Excel today run unchanged inside DNA Calc against CalcDb. Files go both ways: Access opens what CalcDb writes, Compact & Repair finds nothing to fix, and anything CalcDb does not understand survives a rewrite byte for byte. The core also runs off Windows, including in DNA Calc's browser build.

We are not designing a database; we are rebuilding one whose behaviour is fixed by twenty-five years of files, queries and add-ins. Success is how often we match ACE, not what we invent.

In and out

- **In:** Jet 4 `.mdb` and ACE `.accdb`; all Access SQL, including crosstabs, `PARAMETERS`, `DISTINCTROW`, action queries, DDL and both wildcard modes; QueryDefs in Access's own catalog form; ACE's results, types, Null handling, sort orders and error numbers; full DAO, then ADO from VBA (including `Microsoft.ACE.OLEDB.12.0` strings); DAO's real COM interfaces, side by side with ACE. Across processes: correct exclusion, and many readers with no writer.
- **Later:** reading beside an ACE writer, then full shared writing through the lock-file protocol; encryption, linked tables, complex fields, our own compact and repair, OLE DB and ODBC.
- **Out:** running the Access front end (forms, reports, macros and modules are preserved and readable, never run), Jet 3, server roles, and any SQL dialect of our own.
- **Sources:** public documents, Jackcess (Apache 2.0, ported with a licence ledger), mdbtools (facts only) and observed Access/ACE behaviour; never decompiling. FrankenSqlite and asupersync only through reviewed clean-room notes.

Principles

1. **ACE is the specification; the oracle is the judge.** Every behaviour is settled by asking real ACE and Access on Windows; answers are kept as captures keyed by input. A differential harness runs the same SQL, DAO calls and file operations against both and compares rows, types, errors and bytes. A refusal is a failure in every count. The file corpus is a product in its own right.
2. **ISAM first, SQL second.** Jet is a cursor engine with SQL on top, and DAO code leans on `Seek`, bookmarks and table-type recordsets. Tables, indexes, cursors and transactions form one layer, shared by DAO, ADO and SQL.
3. **The file is the truth.** No side format and no write-ahead log of our own: every commit leaves a valid Access file, so write ordering is designed, not incidental. Index keys are byte-identical to ACE's, legacy sort orders included, or Access reports damage. What we do not model we carry as opaque bytes.
4. **Describe the format once.** Page layouts, table definitions, usage maps and catalog rows are written down as data, and reader, writer, validator, fuzzer and page inspector are all driven from it, so they cannot drift apart. Each rule cites its source.
5. **CalcVBA's values, functions and strings.** Basic types (Variant, BSTR strings, Currency, Decimal, Date), conversions and the VBA function library come from CalcVBA, likely via a shared foundation crate, so a value behaves identically in a query and in VBA. CalcVBA holds strings as BSTR to match COM; CalcDb's string design is settled early. ACE's sandbox rules apply. User functions reach CalcVBA as ACE reaches VBA: through the host or by locating the runtime in process, as investigation shows.
6. **Fast alone, then together.** Most Access work is one user, one process, and FrankenSqlite found that multi-version machinery taxes the single-writer path while disjoint writers still collide on shared tree pages and the allocator. Exclusive mode is the fast path, measured first, then in-process snapshots. Across processes the steps are: exclusion that is correct both ways, many readers with no writer, reading beside an ACE writer, and full sharing. One locking interface serves all four from day one.
7. **Deterministic by construction.** Time, I/O, scheduling and randomness reach the engine only through injected interfaces. A simulator replays any interleaving from a seed, injects faults, and crashes the engine at every write point to check the file afterwards. Concurrency bugs are found by search, not by luck.
8. **Blocking calls, owned work.** Object-model calls block the caller's thread, as VBA expects; no async runtime is needed. Inside a call, worker threads from the engine's own scheduler may help where measured, with results equal to serial execution (row order, Double sums); no more parallel than ACE needs. VBA user code runs only on the caller's thread, never under an internal lock. Background work has an owner, a budget and a cancellation protocol; locks and cursors settle when their owner closes.
9. **Portable core, COM everywhere.** The format, ISAM, catalog, expression and SQL crates do no I/O and build for wasm32. The object model is COM interfaces on every platform: `IUnknown`, `IDispatch` and DAO's real IIDs and dispids. Off Windows it runs on the minimal, portable COM stand-in that CalcVBA defines (activation, allocation, BSTR, VARIANT, SAFEARRAY; no apartments or marshaling); CalcDb adds requirements, not an approach of its own.
10. **Explainable and measured.** Plans are inspectable, the plan shown is the plan run, and tests assert plan shape (seek versus scan) as well as results. Performance work has budgets per path, noise control and a ledger of what failed. A ledger built from oracle results shows which DAO members, SQL features and types match ACE.
11. **Evidence over ceremony.** Working code and clear tests beat process apparatus. One of everything: one format description, one ISAM, one evaluator, one oracle, one harness.

Next

Order of work: oracle, corpus, read path → exclusive writing and exclusion → `SELECT` → action queries, DDL, QueryDefs → DAO (COM, stand-in, .NET) → ADO → many readers, in-process sharing → reading beside ACE → full sharing → the rest. **Open:** string design; shared types crate; how UDFs reach CalcVBA; COM activation for add-ins; a host interface; a private undo file for crash safety; the oracle's builds.